KOÇ UNIVERSITY

COMP 350
Introduction to DevOps

Lecture 10
Infrastructure as Code (IaC) and Configuration Management

Hakan Ayral

Infrastructure as code

- Infrastructure as code (**IaC**) is the process of managing and provisioning computer data center resources through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools.
- The IT infrastructure managed by this process comprises both physical equipment, such as bare-metal servers, as well as virtual machines, and associated configuration resources.
- The definitions may be in a version control system, rather than maintaining the code through manual processes.
- The code in the definition files may use either scripts or declarative definitions, but IaC more often employs declarative approaches.

Advantages of IaC

- The value of **IaC** can be broken down into three measurable categories: **cost**, **speed**, and **risk**.
  - **Cost** reduction aims at helping not only the enterprise financially, but also in terms of people and effort, meaning that by removing the manual component, people are able to refocus their efforts on other enterprise tasks.
  - Infrastructure automation enables **speed** through faster execution when configuring your infrastructure and aims at providing visibility to help other teams across the enterprise work quickly and more efficiently.
  - Automation removes the **risk** associated with human error, like manual misconfiguration; removing this can decrease downtime and increase reliability.
- These outcomes and attributes help the enterprise move towards implementing a culture of **DevOps**, the combined working of development and operations.

Types of approaches

- There are generally two approaches to IaC: **declarative** (functional) vs. **imperative** (procedural).
  - The difference between the declarative and the imperative approach is essentially 'what' versus 'how' .
- The **declarative** approach focuses on what the eventual target configuration should be; the **imperative** focuses on how the infrastructure is to be changed to meet this.
- The **declarative** approach defines the desired state and the system executes what needs to happen to achieve that desired state.
- **Imperative** defines specific commands that need to be executed in the appropriate order to end with the desired conclusion.

Continuous configuration automation

- All continuous configuration automation (CCA) tools can be thought of as an extension of traditional IaC frameworks.
- They leverage IaC to change, configure, and automate infrastructure, and they also provide visibility, efficiency and flexibility in how infrastructure is managed.
- These additional attributes provide enterprise-level security and compliance.
- Notable CCA tools include:

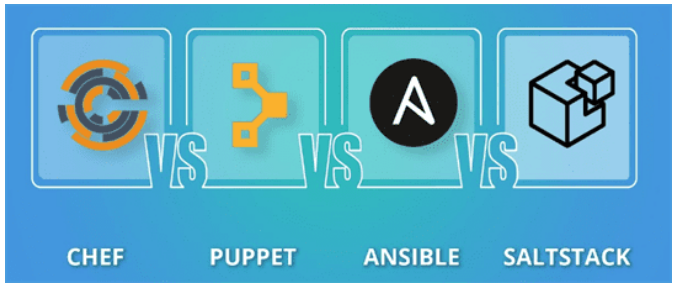
Tool	Released by	Method	Approach	Written in
CFEngine	Northern.tech (1993)	Pull	Declarative	C
Puppet	Puppet (2005)	Push and Pull	Declarative and imperative	C++ & Clojure since 4.0, Ruby
Chef	Chef (2009)	Pull	Declarative and imperative	Ruby
SaltStack	SaltStack (2011)	Push and Pull	Declarative and imperative	Python
Ansible / Ansible Tower	Red Hat (2012)	Push and Pull	Declarative and imperative	Python
Terraform	HashiCorp (2014)	Push	Declarative and imperative	Go
Otter	Inedo (2015)	Push	Declarative and imperative	-

Relationship to DevOps

- IaC can be a key attribute of enabling best practices in DevOps.
- **Developers** become more involved in defining configuration and **Ops teams** get involved earlier in the development process.
- Tools that utilize IaC bring **visibility** to the state and configuration of servers and ultimately provide the visibility to users within the enterprise, aiming to bring teams together to maximize their efforts.
- **Automation** in general aims to take the confusion and error-prone aspect of manual processes and make it more efficient, and productive. Allowing for better software and applications to be created with flexibility, less downtime, and an overall cost-effective way for the company.
- IaC is intended to reduce the complexity that kills efficiency out of manual configuration.
- Automation and collaboration are considered central points in DevOps; infrastructure automation tools are often included as components of a DevOps toolchain.

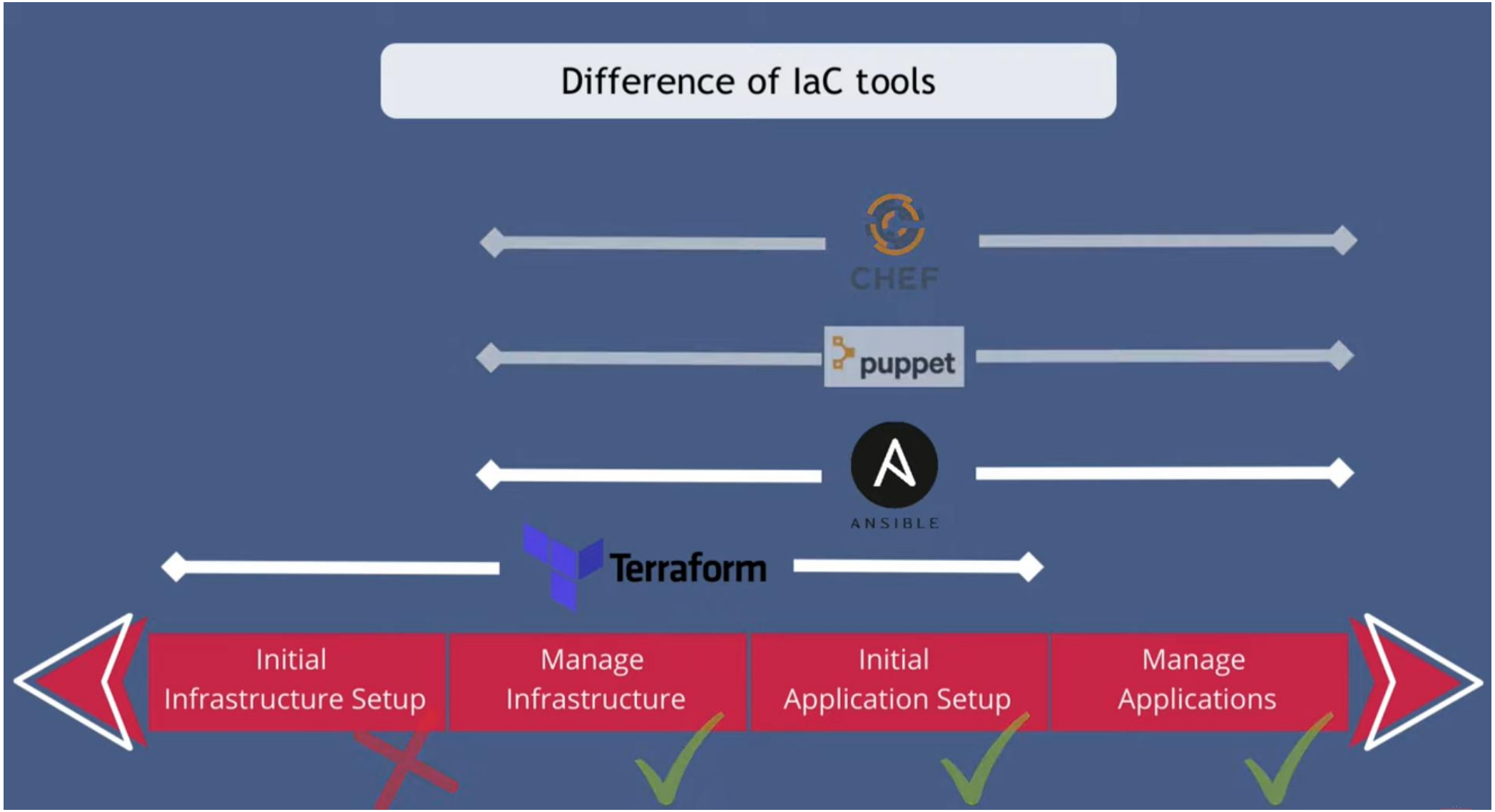
Some open source configuration management software

	Language	License	Mutual auth.	Encryption	Verify mode	Agent-less	Incl. GUI	First release	Latest stable release
Ansible	Python	GPLv3+	Yes[a]	Yes[b]	Yes	Yes	Yes[1]	2012-03-08	2023-11-11 2.15.4[2]
Chef	Ruby, Erlang	Apache 2.0	Yes[f]	Yes[g]	Yes[h][7]	No	Yes	2009-01-15 0.5.0	2023-01-05 18.1.0 (client),[8] 15.4.0 (server)[9]
Puppet	Ruby, C++ & Clojure (server-side also Ruby before 4.0[36])	Apache since 2.7.0, GPL before then	Yes[j]	Yes[d]	Yes[k][37]	No	Yes[38]	2005-08-30[39]	2024-04 8.6.0, 7.30.0 (client),[40] 2024-04 8.6.0, 7.17.0 (server)[41]
Salt[61]	Python[62]	Apache 2.0[63]	Yes[64]	Yes[64]	Yes	Both[65][66]	Yes[67][68]	2011-03-17 0.6.0[69]	2023-05-05 v3006.1[70]



	Ansible	Chef	Puppet	SaltStack
Programming language	Python	Ruby, Erlang	C++, Clojure	Python
Configuration language	YAML, JSON	Ruby	Proprietary	YAML
Database	n/a	PostgreSQL	PuppetDB	n/a
Transport	ssh	RabbitMQ	Mcollective	ZeroMQ
Deployment method	push	pull	pull	push
Master	A, B, L, O, S	Linux	Linux	L, B
Agent	n/a	A, B, L, O, S, W	A, B, L, O, S, W	A, B, L, O, S, W
Agentless	yes	no	no	yes
Public Cloud	AM	AM/AZ/PR	no	no
Cloud management	AM, AZ, OS, GCP	Fog driver	AM, AZ, VM, GCP	Salt Cloud
Architecture	server	client, server	client, server	client, server
Step-by-step	yes	yes	no	no
Enterprise GUI	Ansible Tower	Opscode Manage	Puppet Enterprise	SaltStack Enterprise
Opensource GUI	Semaphore	Chef Manage	Foreman	Saltpad, Saltshaker
Github Stars	25,283	4,979	4,628	8,080
Github Forks	8,790	2,074	1,901	3,762
Contributors	2,957	519	455	1,885
Commits	32,760	19,692	25,816	85,485
Enterprise version	yes	yes	yes	yes

A,B,L,O,S,W : IBM AIX, BSD, Linux, MacOSX, Solaris, Windows


Difference of IaC tools
CHEF
puppet
ANSIBLE
Terraform
Initial Infrastructure Setup
Manage Infrastructure
Initial Application Setup
Manage Applications

Comparison: Database

- Both **Chef** and **Puppet** require a specific database in their architecture: PostgreSQL (**Chef**) and PuppetDB (**Puppet**).
- The **advantage** of using a database is that all configuration files are stored centrally, which allows for – among other things – separate backup of these files.
- The **disadvantage** is that performance and scalability of the entire solution is impacted by an additional component: the **database**.
  - **Chef**, for example, warns not to store the data on network storage, because of potential performance problems.
  - A database requires additional (specialized) maintenance.
- On the other hand, the best practice for storing playbooks in **Ansible** basically is a specific directory layout, which might not suffice either.
  - The enterprise version of Ansible (Tower) does require PostgreSQL, but also MongoDB for a High Availability architecture.

Comparison: Transport

- All the tools use their own way of transport (to get the configuration from master to node).
- **Ansible** uses a secure shell (SSH, Linux) or Powershell (Windows) to transfer configuration, which means that theoretically any device that can run a form or version of SSH can be controlled by Ansible.
- The other three tools use their own specific message broker.

Comparison: Master

- **Ansible** has the widest range of master support: the master or server software is supported on IBM AIX, BSD, Linux, MacOS and Solaris, even though theoretically it could run on any operating system that supports Python 2.6 or 2.7 (3.5 support in tech preview).
- **SaltStack** runs on Linux and BSD.
- **Puppet** and **Chef** run on Linux (RHEL, SLES and Ubuntu in their latest versions) only.

Comparison: Agents

- **Ansible** supports the widest range of agents: as stated above, as long as a client or device can run a secure shell (or Powershell), it can be controlled by Ansible.
  - Of course, control software (in the form of modules) for such a device needs to be available, or written from scratch.
  - On the networking front, apart from supporting traditional Linux based operating systems in such devices, it also supports Cisco IOS and Juniper JunOS natively.
- The three other tools use their own proprietary agents, but such agents are available for all common computing platform.
- Some network equipment is also supported: in **SaltStack**, a proxy-based configuration allows for controlling a network device which has no supported client.
- **Puppet** requires its enterprise version for controlling network devices (such as Cisco NX-OS), whereas **Chef** allows the open source version of its software to do the same.

Comparison: Agentless

- Of the four tools, **Ansible** is the one working completely agentless.
- It is possible, however, to use **SaltStack** in agentless mode as well.
  - This requires SSH and Python on the remote machine.
  - According to SaltStack, this is substantially slower than administration over ZeroMQ (even though no proof is supplied).

Comparison: Public Cloud

- Two out of four tools offer a Public Cloud version of their software:
  - **Ansible** has a version on Amazon,
  - whereas **Chef** has a version for Amazon, Azure and a proprietary cloud hosted by themselves.
- This of course does not mean that **Puppet** and **SaltStack** cannot be hosted in a Public Cloud.

Cloud management

- All four tools offer a form of Cloud management, which means it is possible to use the tool to manage for example Cloud compute nodes.
- The approach differs:
  - **Ansible** offers connectivity to Amazon, Azure, Google Cloud Platform and OpenStack.
  - **Puppet** offers connectivity to Amazon, Azure, Google Cloud Platform and VMWare.
  - Both **Chef** and **SaltStack** offer proprietary Cloud drivers which enables the configuration of various Clouds.

Comparison: Step by step

- Of the four tools, **Ansible** and **Chef** use a step-by-step deployment method.
- Step-by-step in this context means that the actions defined in an **Ansible Playbook** or **Chef Cookbook** are executed line by line, in a so-called **procedural (impreative) configuration**.
- **Puppet** and **SaltStack** use a **declarative configuration**, which means the end-result is defined once and it is up to the tool to figure out the best way to reach this result.

Interlude – Secure Shell (SSH)



- The Secure Shell Protocol (SSH Protocol) is an encrypted network protocol for operating network services securely over unsecured networks.
- SSH was designed for Unix-like operating systems as a replacement for the unencrypted alternatives like **Telnet** and **Berkeley Remote Shell - rsh** (and the related rlogin and rexec protocols).
- Finnish computer scientist Tatu Ylönen designed **SSH** in 1995 and provided an implementation in the form of two commands, **ssh** and **slogin**, as secure replacements for **rsh** and **rlogin**, respectively.
- SSH applications are based on a client–server architecture, connecting an **SSH client** instance with an **SSH server**.
- SSH operates as a layered protocol suite comprising three principal hierarchical components:
  - the **transport layer** provides server authentication, confidentiality, and integrity
  - the **user authentication protocol** validates the user to the server
  - the **connection protocol** multiplexes the encrypted tunnel into multiple logical communication channels.

SSH connection

- For Ansible (and almost any system administration task as part of DevOps) we are going to need an SSH connection from the **control node** to the servers that we are going to manage with Ansible. Therefore let's have a quick review of basic SSH usage.
- Linux typically uses the **OpenSSH** client. To log in to a remote computer with domain name sample.ssh.com, you type the following command at a shell prompt:

```
ssh sample.ssh.com
```

- The usage above assumes that you want to use the same user name as you use on your current machine. To use a different username at the remote machine enter the command as:

```
ssh alternative-username@sample.ssh.com
```

Interlude – SSH single command

- The ssh command is often also used to remotely execute commands on the remote machine without logging in to a shell prompt. The syntax for this is:

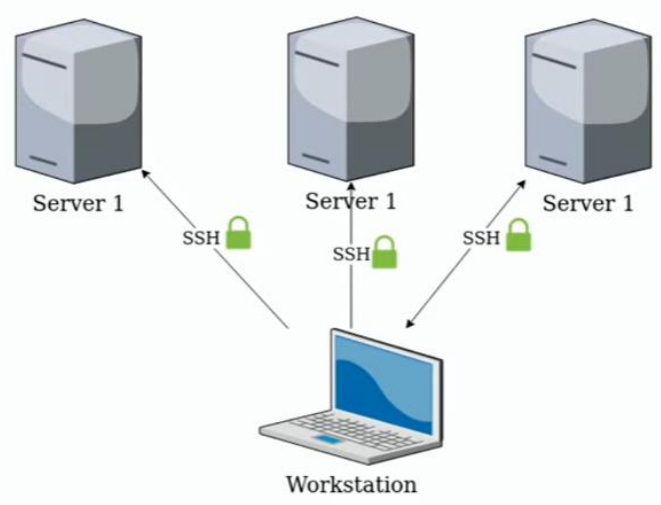
```
ssh hostname command
```

- For example, to execute the command **`ls /tmp/doc`** on host sample.ssh.com, type the following command at a shell prompt:

```
ssh sample.ssh.com  ls /tmp/doc
```

Interlude – SSH server installation

- There are two authentication methods with SSH
  1. **Password** based authentication
  2. **Key** based authentication
     - furthermore the keys can also have a passphrase of their own
- There are multiple different ssh implementations, the most popular one is the **OpenSSH** package.
  - Make sure that you have it installed on workstation and servers for Ansible to be able to function.
  - OpenSSH consists of a client and a server. OpenSSH client is already installed by default on most Linux distributions including Debian/Ubuntu derivatives.
  - OpenSSH server is generally not installed, You can install OpenSSH server as follows:




```
hakan@hakan-VirtualBox:~$ sudo apt-get install openssh-server
[sudo] password for hakan:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  bridge-utils cpu-checker ibverbs-providers ipxe-qemu ipxe-qemu-256k-compat-ef
  libgfrpc0 libgfxdr0 libglusterfs0 libibverbs1 libiscsi7 libndctl6 libpmem1 li
  libspice-server1 libusbredirparser1 libvirglrenderer1 msr-tools ovmf pass qem
  qemu-system-gui qemu-system-x86 qemu-utils qrencode seabios ubuntu-fan uidmap
```

Interlude – first SSH connection

- The first connection between a pair of hosts is different from later connections. On the first connection it asks you whether you want to accept the fingerprint of the remote computer and add it to your list of trusted hosts.

```
hakan@hakan-VirtualBox:~$
hakan@hakan-VirtualBox:~$ ssh 10.0.2.15
The authenticity of host '10.0.2.15 (10.0.2.15)' can't be established.
ED25519 key fingerprint is SHA256:BK4JxHqd54II5JJn+/N+xod2rQcIhEE8vWl5nkmsLO4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.15' (ED25519) to the list of known hosts.
hakan@10.0.2.15's password:

Last login: Wed Mar 13 12:54:09 2024
hakan@hakan-VirtualBox:~$
```

- That's why it is advised that you perform the first connections from the Ansible control node to the servers manually and accept the prompt «Are you sure you want to continue connecting?»
- If you use **password based authentication** it is going to ask you the password for the specified user's password on the remote host.
  - reminder as long as you do not specify a user ssh assumes that your user name is the same on both machines but the password it is asking for is the password for that user on the remote host and not on the local host.

Interlude – SSH key based authentication

- The accepted best practice for SSH connections is to use **key based authentication** instead of password based authentication.
- In order to use key based authentication first you need to **create a key pair** for SSH.
  - SSH uses **public key/private key pairs** (as in asymetric cryptography) instead of passwords when key based authentication is used.
  - Keys can have a **passphrase** of their own, which different from the password of the local or remote host
    - a passphrase for a key is a password that you need to be able to use the key
    - a passphrase is optional for a key, you can leave it blank and as a matter of fact for Ansible you must create a key pair without a passphrase.
  - You have to copy the **public key** part to all the servers you want to connect to and manage with Ansible.
    - Keep the **private key** part safe!
    - There is a special CLI tool which helps adding keys to remote hosts: **ssh-copy-id**

Interlude – generate SSH key pair

- To generate a key pair you can use **ssh-keygen** command:
  - Command option **-C** is for adding a comment to your key (it is just metadata, like a note)
  - You can use command option **–t** to use different types of encryption algorithms for your key

```
hakan@hakan-VirtualBox:~$ ssh-keygen -C "Key for Ansible"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/hakan/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/hakan/.ssh/id_rsa
Your public key has been saved in /home/hakan/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:OW93c+DZHj6wdDsFxXLmeS0UnwkGk4+GuuJ/+y0ZJkc Key for Ansible
The key's randomart image is:
+---[RSA 3072]----+
|           ooo.o |
|           .o +.B|
|          . o. O+|
|         o E .o.+|
|        S o   .o.|
|       . + + + =.|
|        . * = O.=|
|     . . o +.o.B.|
|    ..o...o... .+|
+----[SHA256]-----+
hakan@hakan-VirtualBox:~$
```

Interlude – generate SSH key pair

- The generated key pair is stored in the directory **.ssh** on your home directory as two files (one file for public key and one file for private key)

```
hakan@hakan-VirtualBox:~$
hakan@hakan-VirtualBox:~$ ls -la .ssh
total 30
drwx------  2 hakan hakan    6 May 13 12:25 .
drwxr-x--- 20 hakan hakan   40 May 13 12:06 ..
-rw-------  1 hakan hakan 2602 May 13 12:25 id_rsa
-rw-r--r--  1 hakan hakan  569 May 13 12:25 id_rsa.pub
-rw-------  1 hakan hakan  978 May 13 12:06 known_hosts
-rw-r--r--  1 hakan hakan  142 May 13 12:06 known_hosts.old
hakan@hakan-VirtualBox:~$
```

- Their contents look like this; Private key

```
hakan@hakan-VirtualBox:~$ cat .ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEA3HfwVwy28zWw1wuISW5snTFDiTSDnR8P2tubJTboh9Q5qXT0Zblp
n4jeAEWKLgroohYyKyXrf/LsSTEeeOfs6F7hU8WIXoqHvtqiODY1DUY4GG4KiJe9PzSQ8N
KWuVTUAyk1nN4/rfxGKD/pZ5bXuyWye6223F7Ah4Nvqv68B5HutpqZzST0INez13vRyx3F
LhCh2Xtb1BtZnzihGYvhkSDOe+28nBtN2TVqvFZ7H7oyOjyanHsH5WkMGcPjeKjbXtmS0A
4Ilv6DvUEIkmS1+/XwFMB7GZK9SWcATXFJOvjBJKQloh4EwGuZn+KV1AjVznNXuycx9mwB
CIaSLReJ5gOHETF3ZpRrmoeFDAQvQc2efAPP9lC5sreAdhw92J31mAiydkYwBGBwmiEWY3
T3z0FCUs/AK2L7f0EQKv6HTpbT6KbbczGgwQZD31aULTxUGzIbX34v5rYU2L7IKDO9JGji
pUuvVDWAdBZoN6X69FcZOkyuFtPrAfF66kwfymcxAAAFiLkj4B65I+AeAAAAB3NzaC1yc2
EAAAGBANx38FcMtvM1sNcLiElubJ0xQ4k0g50fD9rbmyU26IfUOal09GW5aZ+I3gBFii4K
6KIWMisl63/y7EkxHnjn7Ohe4VPFiF6Kh77aojg2NQ1GOBhuCoiXvT80kPDSlrlU1AMpNZ
zeP638Rig/6WeW17slsnutttxewIeDb6r+vAeR7raamc0k9CDXs9d70csdxS4Qodl7W9Qb
WZ84oRmL4ZEgznvtvJwbTdk1arxWex+6Mjo8mpx7B+VpDBnD43io217ZktAOCJb+g71BCJ
Jktfv18BTAexmSvUlnAE1xSTr4wSSkJaIeBMBrmZ/ildQI1c5zV7snMfZsAQiGki0XieYD
hxExd2aUa5qHhQwEL0HNnnwDz/ZQubK3gHYcPdid9ZgIsnZGMARgcJohFmN0989BQlLPwC
ti+39BECr+h06W0+im23MxoMEGQ99WlC08VBsyG19+L+a2FNi+yCgzvSRo4qVLr1Q1gHQW
aDel+vRXGTpMrhbT6wHxeupMH8pnMQAAAAMBAAEAAAGAAZquUSXWqXWjLidndBhVej3W5t
dO21xz4DTvn7FS9xUOX9sp4ZkcKYTkj6Yx45Sg/XfxoMJ/gJB3E2/D79+JJ+Yz18kUzVJO
+06qNkQ/tdNkWlonobeP8Rcr9FKl9d6GBeldemEcH8MZjBat9zrf4jzbm/6HOXqHhlzomb
sNiXUQgnvkwVtC2v/Sz083dzXF3uhQUy2sfQI/cFMIShAsYaLWWczArfadZgaXWpw6/F+9
mJKjEd8rzymo5MGaBMFexs+JoHqzg2L7hXPzy1yVCOI/zaxQUZlET8xsRV3H64ZItqB5dC
faoN90Ehb6A+ltUtGXBcGPRZrhp3vPVl80/XzAE0nAOAXS+H55p1pl0dIn8XwtwbZh3qBj
S7HkMBY4LJU60yX0HjEHM1LhnXzOQeodcrWXDQNQ46LCa5OdMcLob1ZETaC8r0+CMj45YO
V2YhOCoVZgigO1R6jXWVOkqeiTj4MEjwfEi09VunGD9hSZgBCOp2EZtE5gTMJAC0SBAAAA
wH9On1/ec2gwZSDk44IofWszmgcPvuktMdhE3Lr+t/Vb4fjjvm3gmKGhc0bpNZpzXMjuZ9
Xhh0yZqYMUFEbRHh/xXh0HCrTZJxnx1t7Bc2TmpgfsIcCHwpodtpYSsfoRpN/CQNkZ9i2m
mQ/k3qtpFgz+G76n0wLOQaGqojFwRMFJ26AXFG/yNoWowZejrj8Ca9AhkL5gY3ZoWcySfX
62NJ2+F11A8hiLRkYCQBTcaVCMGJse4iqH2pcyV2jrEzrHLQAAAMEA+JBgzRiVurGorVek
/kV9La7/TBYPxy+22E2X/86TE9Gt6HzDC1Opvzi+Q3oQNYosyJVs8cyvGzj1BvVYagOPZ4
GCNk/gS47HpIDwu1rhlKeqpDisBNuMsyME5ANY4ZzzyzYcHixe7SHrSXkjkxm65fWjalw5
THT32RhCb/XDy/+Ql3H6lgBodnncRy2SxwFXGwySpErwmArZXFJLlz5EVMw5SsyViVhlCj
6cObW5EnjRcWPdYD8iOOmyH9XMyUhBAAAAwQDjEGRlsXoPgKRXH2ik5aK2o3FlW0776Vjx
rYjk/knRa0JBWq0SOt6smrJFO3BoPSzRCSZj2VJi3ha/fVMp7IdOqw19oix3IdPWHQv8Wg
dNNAlf4Y6yPRmq9VrYyoNodW0A4SGVFS2MdnnaTwCBnoujxwfil8Hd3EjKpqjpYmcbu7qh
SXiTFpz88KdXg92Rh85XxiLh+2qUhm+P8/bOS7SBX+Hf2zIxOP+5oHunYNdWZPshqBPIoT
h+56bl8qOE4vEAAAAPS2V5IGZvciBBbnNpYmxlAQIDBA==
-----END OPENSSH PRIVATE KEY-----
hakan@hakan-VirtualBox:~$
```

- Public key

```
hakan@hakan-VirtualBox:~$ cat .ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDcd/BXDLbzNbDXC4hJbmydMUOJNIOdHw/a25slNuiH1DmpdPRluWmfiN4ARYouCuiiFjIrJet/8uxJMR545+zoXuFTxYheioe+
2qI4NjUNRjgYbgqIl70/NJDw0pa5VNQDKTWc3j+t/EYoP+lnlte7JbJ7rbbcXsCHg2+q/rwHke62mpnNJPQg17PXe9HLHcUuEKHZe1vUG1mfOKEZi+GRIM577bycG03ZNWq8Vnsf
ujI6PJqcewflaQwZw+N4qNte2ZLQDgiW/oO9QQiSZLX79fAUwHsZkr1JZwBNcUk6+MEkpCWiHgTAa5mf4pXUCNXOc1e7JzH2bAEIhpItF4nmA4cRMXdmlGuah4UMBC9BzZ58A8/2
ULmyt4B2HD3YnfWYCLJ2RjAEYHCaIRZjdPfPQUJSz8ArYvt/QRAq/odOltPopttzMaDBBkPfVpQtPFQbMhtffi/mthTYvsgoM70kaOKlS69UNYB0Fmg3pfr0Vxk6TK4W0+sB8Xrq
TB/KZzE= Key for Ansible
hakan@hakan-VirtualBox:~$
```

type ed25519 is better than RSA ?

Interlude – add the public SSH key to remote

- You have to copy the public key (and only the public key!) to the rmeote host you want to connect to via ssh without using the user password.
- The command to use is `ssh-copy-id –i <file-name-of-public-key> <remote-ip>`
- It will ask the user password of the remote host (*not the key file passphrase*, *not the password of the local user*)

```
hakan@hakan-VirtualBox:~$ ssh-copy-id -i .ssh/id_rsa.pub 10.0.2.15
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ".ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
hakan@10.0.2.15's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh '10.0.2.15'"
and check to make sure that only the key(s) you wanted were added.

hakan@hakan-VirtualBox:~$
```

- Now you can connect to the remote host securely without typing any password ☺

```
hakan@hakan-VirtualBox:~$ ssh 10.0.2.15

Last login: Mon May 13 12:21:22 2024 from 10.0.2.5
hakan@hakan-server-blue:~$
```